Data Structures
Queues



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk



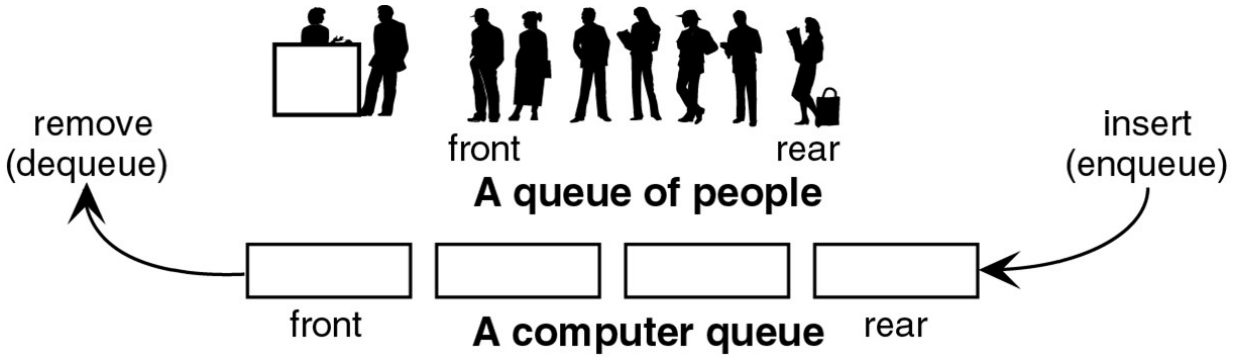


Learning Outcomes

- On completion of this session, you should be able to
  - Define queue data structure
  - Identify the access rules, operations and error conditions associated with a queue
  - Declare array-based and linked list-based queues
  - Implement array-based and linked list-based queues
  - Use queues in solving various real-world problems

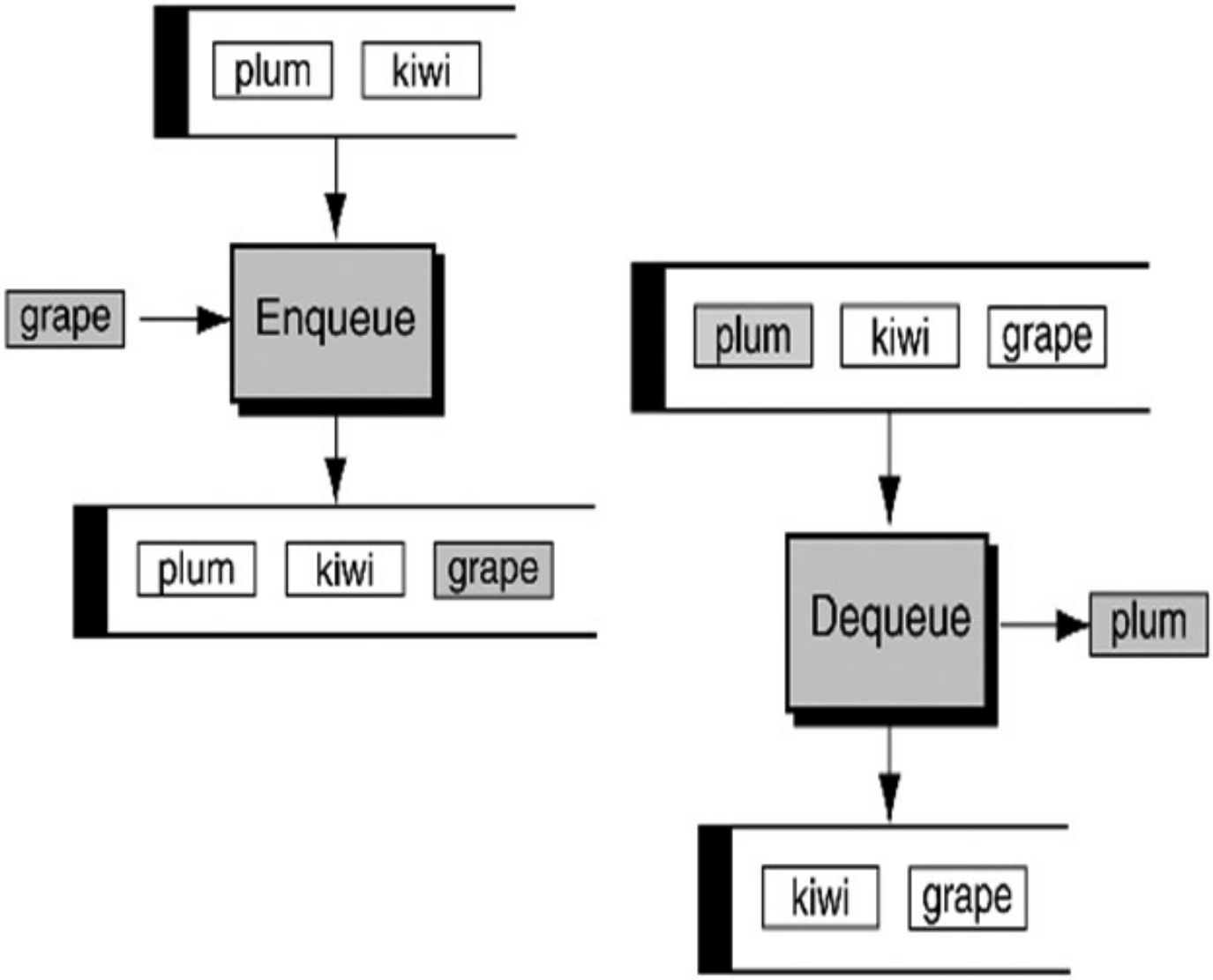
Queue

- A linear data structure
- Data additions are restricted to a one end (rear) and removals can only be done from the other end (front)
- First In First Out (FIFO)
- Error conditions
  - Overflow
    Happens when trying to add an item to a full queue
  - Underflow
    Happens when attempting to remove an item from an empty queue



Operations

- enqueue( )
- dequeue( )

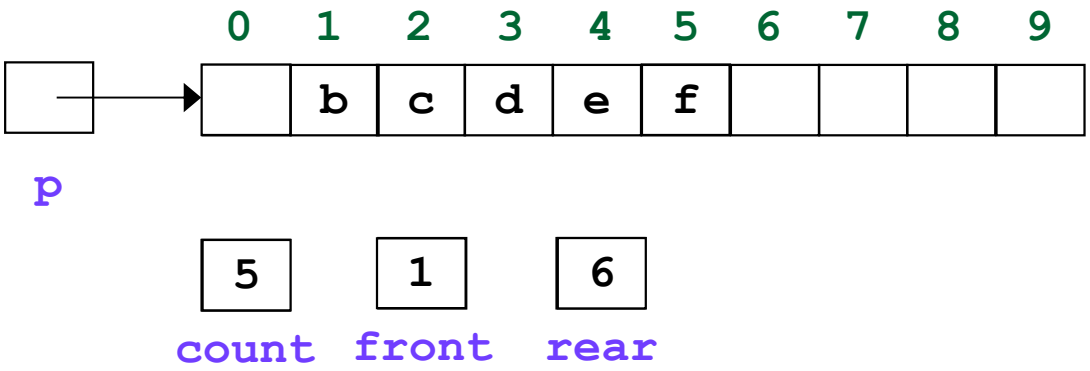


Array-based Queue

- Conceptual Queue
- Physical Queue

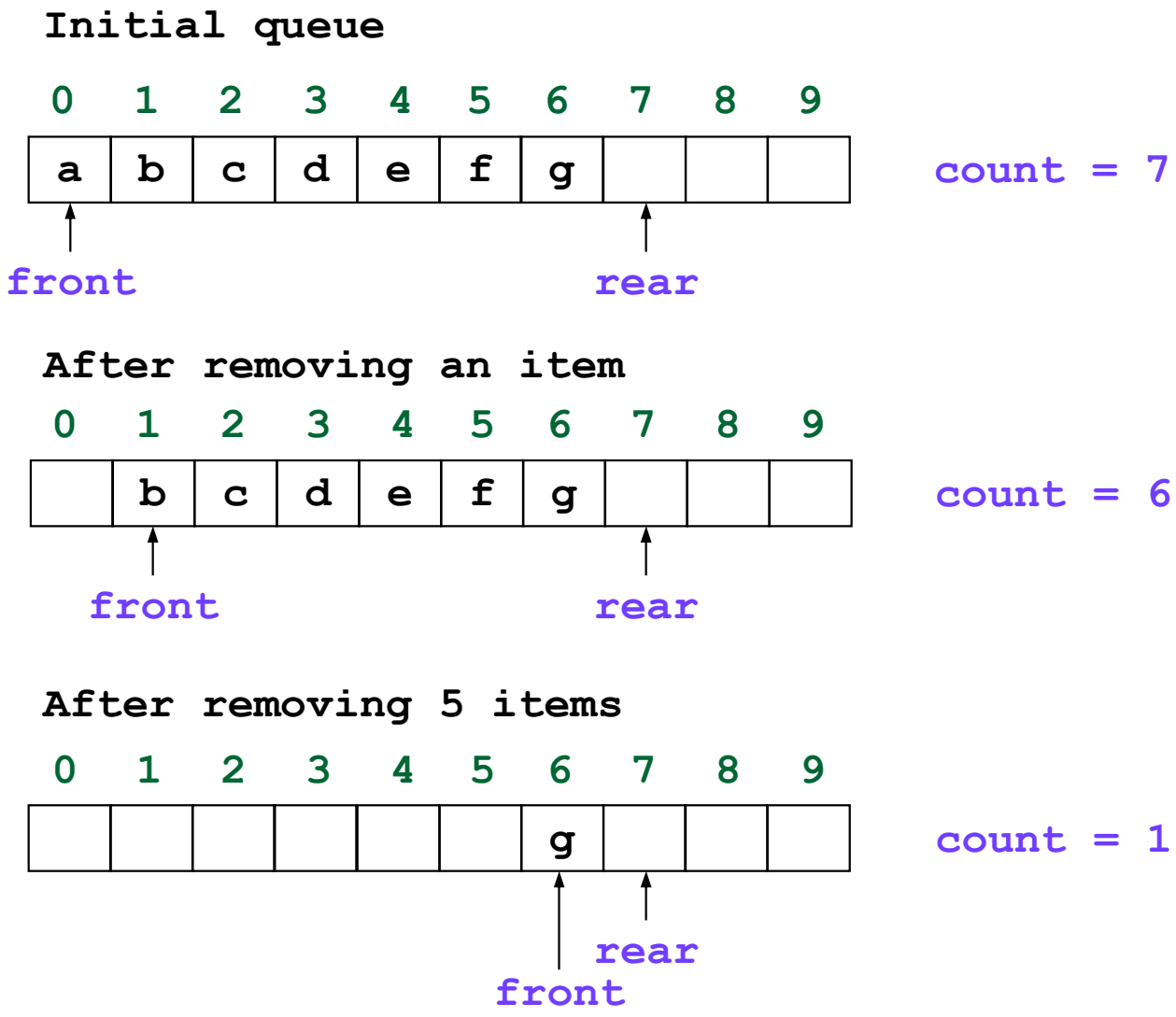


  - Arrays are finite by their nature, but if they were not
    - Front would be the left end and rear would be the right end
    - With insertions and removals, both front and rear would move to the right



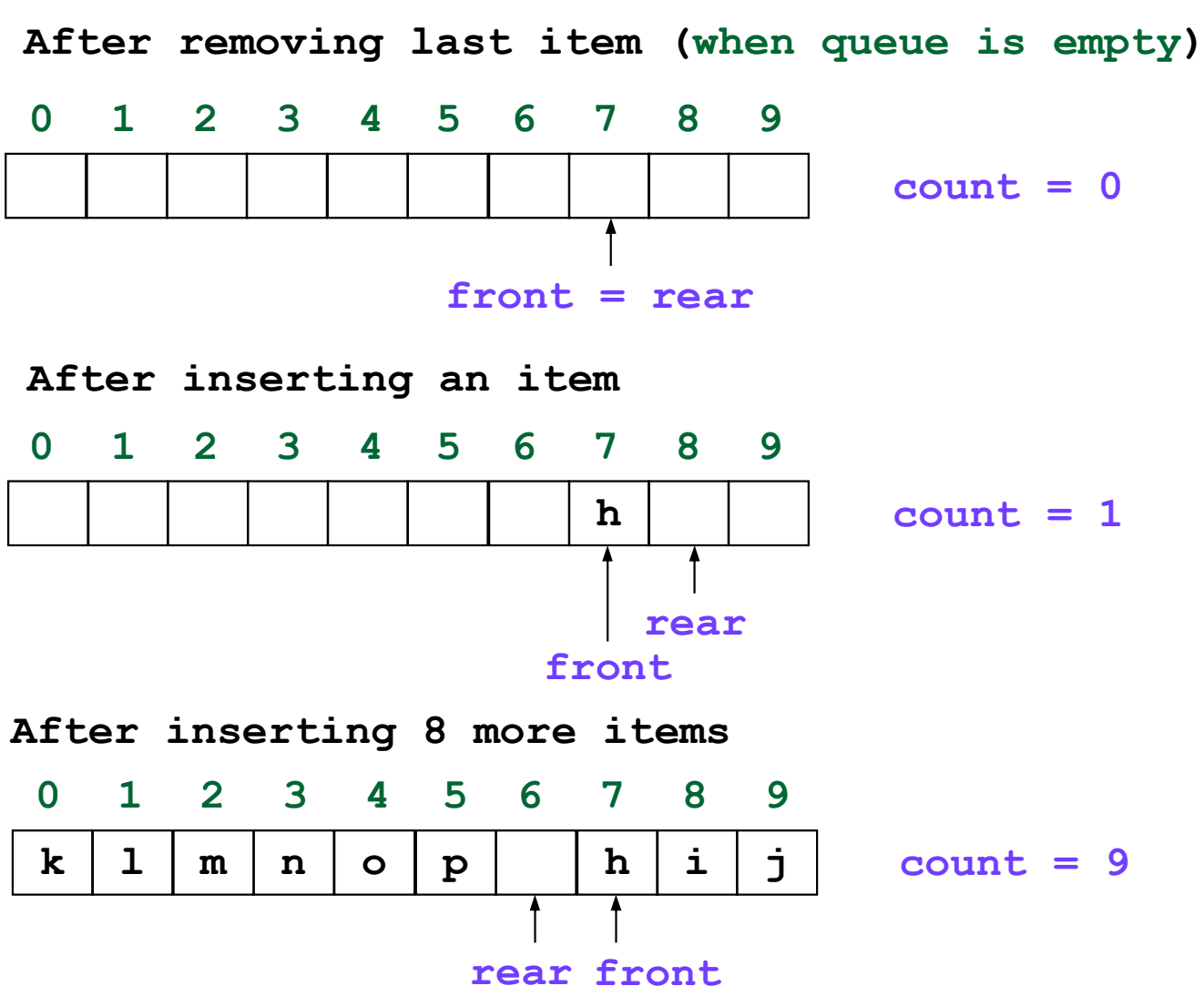
Array-based Queue

- Example



Array-based Queue

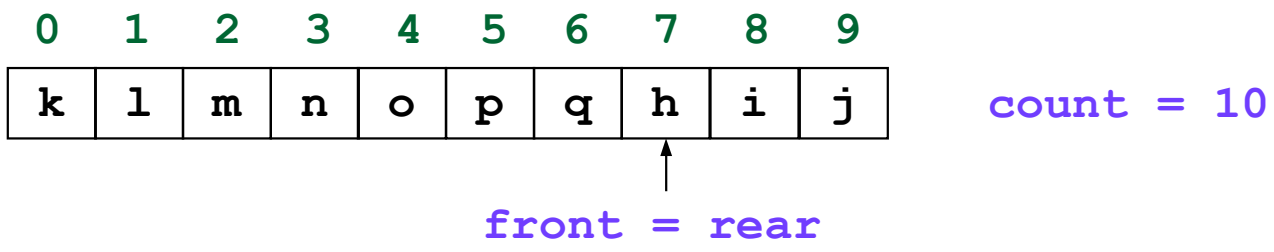
- Example (cont.,)



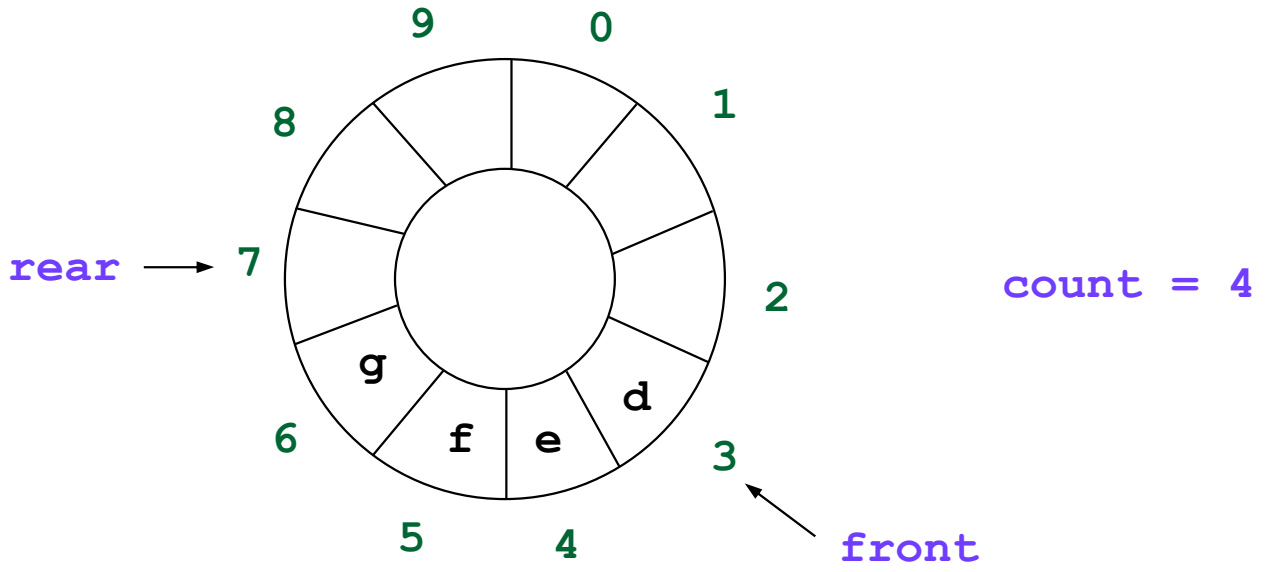
Array-based Queue

- Example (cont.,)

After inserting another items (when queue is full)



Circular array-based queue



Array-based Queue

- Global Variables

```
p POINTER to DATATYPE, size INTEGER, count INTEGER,
front INTEGER, rear INTEGER
```

- create Operation

```
ALGORITHM create(VAL s INTEGER)
1  size ← s
2  front ← rear ← count ← 0
3  p ← ALLOCATE MEMORY DATATYPE[size]
END create
```

- destroy Operation

```
ALGORITHM destroy()
1  RELEASE MEMORY[] p
END destroy
```

Array-based Queue

- isFull Operation

```
ALGORITHM isFull()
1  IF (count = size)
2     RETURN true
3  ELSE
4     RETURN false
5  END IF
END isFull
```

- isEmpty Operation

```
ALGORITHM isEmpty()
1  IF (count = 0)
2     RETURN true
3  ELSE
4     RETURN false
5  END IF
END isEmpty
```

Array-based Queue

- enqueue Operation

```
ALGORITHM enqueue(VAL d DATATYPE)
1  IF (isFull())
2     RETURN false
3  ELSE
4     p[rear] ← d
5     rear ← rear + 1
6     count ← count + 1
7     rear ← rear MOD size
8     RETURN true
9  END IF
END enqueue
```

Array-based Queue

- dequeue Operation

```
ALGORITHM dequeue(REF d DATATYPE)
1  IF (isEmpty())
2     RETURN false
3  ELSE
4     d ← p[front]
5     front ← front + 1
6     count ← count - 1
7     front ← front MOD size
8     RETURN true
9  END IF
END dequeue
```

Array-based Queue

- test Algorithm

```
ALGORITHM test()
1  create(queueSize)
2  READ(data)
3  LOOP(data ≠ endOfData AND enqueue(data))
4     READ(data)
5  END LOOP
6  LOOP(dequeue(data))
7     PRINT(data)
8  END LOOP
9  destroy()
END test
```

Array-based Queue Implementation

```
//ArrayQueue.h header file
class queue
{
   public:
      queue(int); //create() operation
      ~queue(); //destroy() operation
      int enqueue(char);
      int dequeue(char &);
   private:
      int isEmpty();
      int isFull();
      int size, count, front, rear;
      char *p;
};
```

Array-based Queue Implementation

```
queue::queue(int s)
{
   size = s;  front = rear = count = 0;
   p = new char[size]; //allocates memory
}
queue::~queue()
{
   delete[] p; //releases memory
}
int queue::isFull()
{
   return (count == size) ? 1 : 0;
}
int queue::isEmpty()
{
   return (count == 0) ? 1 : 0;
}
```

Array-based Queue Implementation

```
int queue::enqueue(char d)
{
   if (isFull())
      return 0;
   else
   {
      p[rear++] = d;
      count++;
      rear = rear % size;
      return 1;
   }
}
```

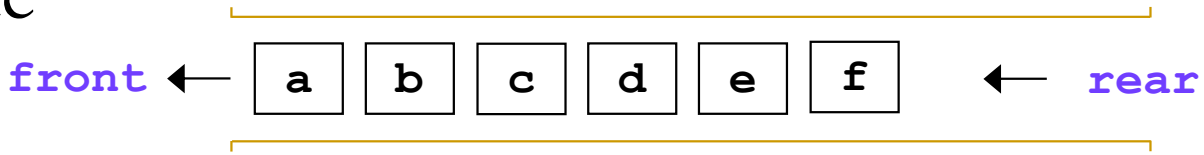
```
int queue::dequeue(char &d)
{
   if (isEmpty())
      return 0;
   else
   {
      d = p[front++];
      count--;
      front = front % size;
      return 1;
   }
}
```

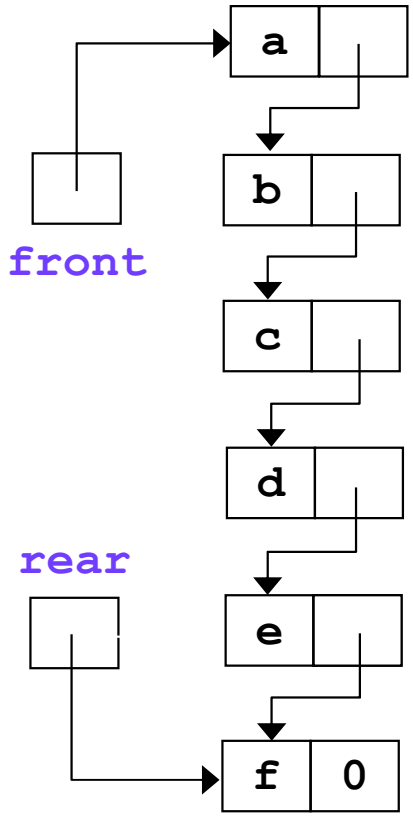
Array-based Queue Implementation

```
//TestArrayQueue.cpp source file
#include <iostream>
#include "ArrayQueue.h"
int main()
{
   const int queueSize = 10;
   queue q(queueSize);
   char data;
   std::cin >> data;
   while(data != '.' && q.enqueue(data))
      std::cin >> data;
   std::cout << std::endl;
   while(q.dequeue(data))
      std::cout << data;
   return 0;
}
```

Linked List-based Queue

- Conceptual Queue
- Physical Queue
  - After inserted an item
  - After removed an item





Linked List-based Queue

- Global Variables

```
TYPE node
    data DATATYPE
    next POINTER to node
END node

front POINTER to node, rear POINTER to node
```

- create Operation

```
ALGORITHM create()
1  front ← null
2  rear ← null
END create
```

Linked List-based Queue

- destroy Operation

```
ALGORITHM destroy()
1  LOOP(front ≠ null)
2      p ← front
3      front ← front->next
4      RELEASE MEMORY p
5  END LOOP
END destroy
```

- isEmpty Operation

```
ALGORITHM isEmpty()
1  IF (front = null)
2      RETURN true
3  ELSE
4      RETURN false
5  END IF
END isEmpty
```

Linked List-based Queue

- enqueue Operation

```
ALGORITHM enqueue(VAL d DATATYPE)
1  p ← ALLOCATE MEMORY node
2  p->data ← d
3  p->next ← null
4  IF (front ≠ null)
5     rear->next ← p
6  ELSE
7     front ← p
8  END IF
9  rear ← p
END enqueue
```

Linked List-based Queue

- dequeue Operation

```
ALGORITHM dequeue(REF d DATATYPE)
1  IF (isEmpty())
2     RETURN false
3  ELSE
4     d ← front->data
5     p ← front
6     IF (front = rear)
7        front ← rear ← null
8     ELSE
9        front ← front->next
10    END IF
11    RELEASE MEMORY p
12    RETURN true
13 END IF
END dequeue
```

Linked List-based Queue

- test Algorithm

```
ALGORITHM test()
1  create()
2  READ(data)
3  LOOP(data ≠ endOfData)
4      enqueue(data)
5      READ(data)
6  END LOOP
7  LOOP(dequeue(data))
8      PRINT(data)
9  END LOOP
10 destroy()
END test
```

Linked List-based Queue Implementation

```
//LinkedListQueue.h header file
struct node
{
   char data;
   node *next;
};
class queue
{
   public:
      queue(); //create() operation
      ~queue(); //destroy() operation
      void enqueue(char);
      int dequeue(char &);
   private:
      int isEmpty();
      node *front, *rear;
};
```

Linked List-based Queue Implementation

```
queue::queue()
{
   front = rear = 0; //sets to null
}

queue::~queue()
{
   while(front)
   {
      node *p = front;
      front = front->next;
      delete p; //releases memory
   }
}
```

Linked List-based Queue Implementation

```
int queue::isEmpty()
{
   return (front == 0) ? 1 : 0;
}

void queue::enqueue(char d)
{
   node *p = new node; //allocates memory
   p->data = d;
   p->next = 0;
   (front ? rear->next : front) = p;
   rear = p;
}
```

Linked List-based Queue Implementation

```
int queue::dequeue(char &d)
{
   if (isEmpty())
      return 0;
   else
   {
      d = front->data;
      node *p = front;
      if (front == rear)
         front = rear = 0;
      else
         front = front->next;
      delete p;
      return 1;
   }
}
```

Linked List-based Queue Implementation

```
//TestLinkedListQueue.cpp source file
#include <iostream>
#include "LinkedListQueue.h"
int main()
{
   queue q;
   char data;
   std::cin >> data;
   while(data != '.')
   {
      q.enqueue(data);
      std::cin >> data;
   }
   std::cout << std::endl;
   while(q.dequeue(data))
      std::cout << data;
   return 0;
}
```

Queue Applications

- Process scheduling in OS
- Printer Spooling
- Network traffic handling
- Many other data processing and service providing applications

Questions